

Multivariate Analysis - CA7

Regression



Joshua Orelowitz

ORLJOS001

Kai Van Niekerk

VNKKAI001

Alan Hunter

HNTALA001

University of Cape Town

19 April 2026

Table of contents

Question 1	2
a) Regression Coefficients Estimates	2
b) Confidence Interval for the effect of Soil PH on Temperature	3
c) F-test for the effect of soil PH and distance to water on maize yield	4
d) Likelihood ratio test for the effect of soil PH on either maize yield or sunlight . .	5
Question 2	6
Multiple Regression	6
PCR	7
PLS	9
Summary	10

Question 1

a) Regression Coefficients Estimates

i)

```
n = nrow(data)

#design matrix
X = cbind(1, data_tidy_yield$SoilPH, data_tidy_yield$DistanceToWater)
colnames(X) = c("Intercept", "SoilpH", "DistToWater")

#selecting our response variables
Y_mat = as.matrix(data_tidy_yield$CropYield)
colnames(Y_mat) = c("MaizeYield")

#this solves the entire MV case
Beta_hat = solve(t(X) %*% X) %*% t(X) %*% Y_mat

kable(Beta_hat,
      digits = 4,
      caption = "Regression coefficient estimates.",
      col.names = c("Maize Yield")
    )
```

	Maize Yield
Intercept	20.7219
SoilpH	1.8294
DistToWater	-0.0465

Table 1: Regression coefficient estimates.

ii)

```
#selecting our response variables
Y_mat = as.matrix(data_tidy_yield$Sunlight)
colnames(Y_mat) = c("Sunlight")

#this solves the entire MV case
Beta_hat = solve(t(X) %*% X) %*% t(X) %*% Y_mat

kable(Beta_hat,
      digits = 4,
      caption = "Regression coefficient estimates.",
      col.names = c("Sunlight")
    )
```

	Sunlight
Intercept	10.8590
SoilpH	-0.0089
DistToWater	0.0445

Table 2: Regression coefficient estimates.

iii)

```
#selecting our response variables
Y_mat = cbind(data_tidy_yield$CropYield, data_tidy_yield$Sunlight)
colnames(Y_mat) = c("MaizeYield", "Sunlight")

#this solves the entire MV case
Beta_hat = solve(t(X) %*% X) %*% t(X) %*% Y_mat

kable(Beta_hat,
      digits = 4,
      caption = "Regression coefficient estimates.",
      col.names = c("Maize Yield", "Sunlight")
    )
```

	Maize Yield	Sunlight
Intercept	20.7219	10.8590
SoilpH	1.8294	-0.0089
DistToWater	-0.0465	0.0445

Table 3: Regression coefficient estimates.

b) Confidence Interval for the effect of Soil PH on Temperature

```
#using the celsius response for temp
y_b = data$TempC
x_b = cbind(1, data$SoilPH)
n = nrow(x_b)
k = ncol(x_b)

#coeff estimates
beta_b = solve(t(x_b) %*% x_b) %*% t(x_b) %*% y_b

var_resp = sum(t((y_b - x_b %*% beta_b)) %*% (y_b - x_b %*% beta_b)) / (n - k)
```

```

var_beta = var_resp * solve(t(x_b) %*% x_b)

#our variance for soil ph in the covariance matrix above
se_soilph = sqrt(var_beta[2,2])

#get the confidence interval
t_beta = qt(0.975, df = n - k)
ci_lower = beta_b[2] - t_beta * se_soilph
ci_upper = beta_b[2] + t_beta * se_soilph

ci_res = data.frame(
  Estimate = beta_b[2],
  Lower_CI = ci_lower,
  Upper_CI = ci_upper
)

kable(ci_res, digits = 4,
      caption = "95% confidence interval for the effect of soil PH on temperature.")

```

Estimate	Lower_CI	Upper_CI
1.5547	0.791	2.3183

Table 4: 95% confidence interval for the effect of soil PH on temperature.

c) F-test for the effect of soil PH and distance to water on maize yield

```

#need full model results
beta_full = Beta_hat[, 1]
n = nrow(X)
k = ncol(X)
q = 2

y_c = data_tidy_yield$CropYield
var_resp_c = (t(y_c - (X %*% beta_full)) %*% (y_c - (X %*% beta_full))) / (n - k)

#the matrix of our estimates we are testing
beta_1 = matrix(beta_full[2:3], nrow = 2)
C = solve(t(X) %*% X)
C11 = C[2:3, 2:3]

#get f-statistic
f_stat = t(beta_1) %*% solve(C11) %*% beta_1 / (q * var_resp_c)
p_val = pf(f_stat, df1 = q, df2 = n - k, lower.tail = FALSE)

```

```
f_results = data.frame(
  "F_Statistic" = f_stat,
  "Numerator_DF" = q,
  "Denominator_DF" = n - k,
  "P_Value" = p_val
)

kable(f_results, digits = 4,
      caption = "F-test result for the joint effect of Soil pH and Distance to Water.")
```

F.Statistic	Numerator_DF	Denominator_DF	P_Value
29.1032	2	122	0

Table 5: F-test result for the joint effect of Soil pH and Distance to Water.

d) Likelihood ratio test for the effect of soil PH on either maize yield or sunlight

```
Y = cbind(data$CropYield, data$Sunlight)
n = nrow(Y)

#full model error
error_full = Y - (X %*% Beta_hat)
Sigma_MLE = (1/n) * (t(error_full) %*% error_full)

#reduced model
X_reduced = X[, -2]
Beta_reduced = solve(t(X_reduced) %*% X_reduced) %*% t(X_reduced) %*% Y
error_reduced = Y - (X_reduced %*% Beta_reduced)
Sigma_MLE_2 = (1/n) * (t(error_reduced) %*% error_reduced)

ratio = det(Sigma_MLE) / det(Sigma_MLE_2)

k = 3
r = 2
p = 2
q = 1

#the chi-squared approximation
lrt_stat = -(n - k - 0.5 * (r - p + q + 1)) * log(ratio)
p_val = pchisq(lrt_stat, df = r * q, lower.tail = FALSE)

lrt_final = data.frame(
  "Wilks_Lambda" = ratio,
```

```

"Chi_Squared_Approximation" = lrt_stat,
"DF" = r * q,
"P_Value" = p_val
)
kable(lrt_final, digits = 4, caption = "Multivariate Likelihood ratio test.")

```

Wilks_Lambda	Chi_Squared_Approximation	DF	P_Value
0.6712	48.2416	2	0

Table 6: Multivariate Likelihood ratio test.

Question 2

Multiple Regression

```

mali = na.omit(data_tidy_mali)
y_raw = mali$cotton
X_raw = as.matrix(mali[, names(mali) != "cotton"])
n_m = nrow(X_raw)
one_m = matrix(1, n_m, 1)

X_mlr = cbind(one_m, X_raw)
B_mat = solve(t(X_mlr) %*% X_mlr) %*% t(X_mlr) %*% y_raw

```

```

kable(B_mat, digits = 4, caption = "Multiple Regression Beta Coefficients")

```

	-0.4342
family	0.0011
dist_rd	0.0032
maize	0.6445
sorg	0.2111
millet	0.0578
bull	0.5538
cattle	-0.0581
goats	0.0337

Table 7: Multiple Regression Beta Coefficients

```

y_hat <- X_mlr %*% B_mat

SSE <- sum((y_raw - y_hat)^2)

```

```
SST <- sum((y_raw - mean(y_raw))^2)
r2_mr <- 1 - SSE/SST
r2_mr |> signif(3)
```

```
[1] 0.774
```

The multiple regression model achieved an R^2 of 0.774.

PCR

```
#scaling and normalizing
X = scale(X_raw)
y = scale(y_raw)
n = nrow(X)
p = ncol(X)

svd_X = svd(X)
V = svd_X$v

#finding the optimal number of components to include
component_variances = svd_X$d^2
pve = component_variances / sum(component_variances)
```

```
pca_summary <- tibble(
  PC = paste0("PC", 1:p),
  Eigenvalue = svd_X$d^2,
  `Proportion of Variance` = pve,
  `Cumulative Proportion` = cumsum(pve)
)

pca_summary |> kableExtra::kable(
  digits = 3,
  booktabs = TRUE,
  caption = "Eigenvalues and proportion of variance explained by each principal component"
)
```

PC	Eigenvalue	Proportion of Variance	Cumulative Proportion
PC1	263.533	0.439	0.439
PC2	82.638	0.138	0.577
PC3	77.906	0.130	0.707
PC4	68.923	0.115	0.822
PC5	45.028	0.075	0.897
PC6	36.119	0.060	0.957

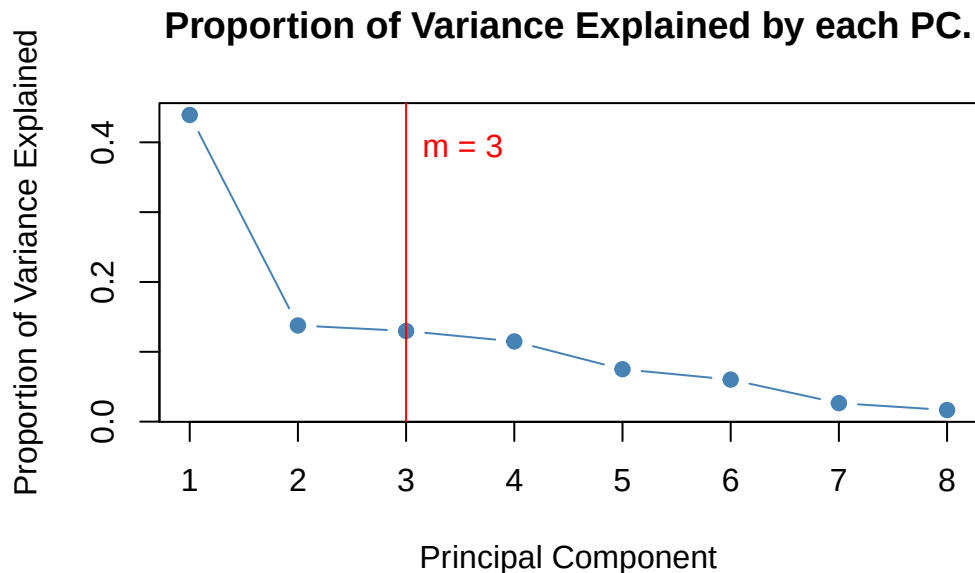
PC	Eigenvalue	Proportion of Variance	Cumulative Proportion
PC7	15.895	0.026	0.983
PC8	9.958	0.017	1.000

Table 8: Eigenvalues and proportion of variance explained by each principal component

The first three principal components explain approximately 70.7% of the total variation, so m is chosen to be 3.

```
plot(pve,
     type = "b",
     pch = 19,
     col = "steelblue",
     xlab = "Principal Component",
     ylab = "Proportion of Variance Explained",
     main = "Proportion of Variance Explained by each PC.")

abline(v = 3, col = "red")
text(x = 3.15, y = max(pve) * 0.9,
     labels = "m = 3",
     col = "red",
     adj = 0)
```



From the scree plot above we have decided to use a cut off of 3 PCs as this will cover most of the variance within our dataset, this supports what we discovered from the table above.

```
#lets set m as 3 components
Z_m= X %*% V[, 1:3]

#pca component coefficients
coeffs = (solve(t(Z_m) %*% Z_m) %*% t(Z_m) %*% y)
beta_pcr = V[, 1:3] %*% coeffs
```

```
#pcr metrics
y_hat_pcr = Z_m %*% coeffs
mse_pcr = mean((y - y_hat_pcr)^2)
r2_pcr = 1 - (sum((y - y_hat_pcr)^2) / sum((y - mean(y))^2))
r2_pcr |> signif(3)
```

```
[1] 0.703
```

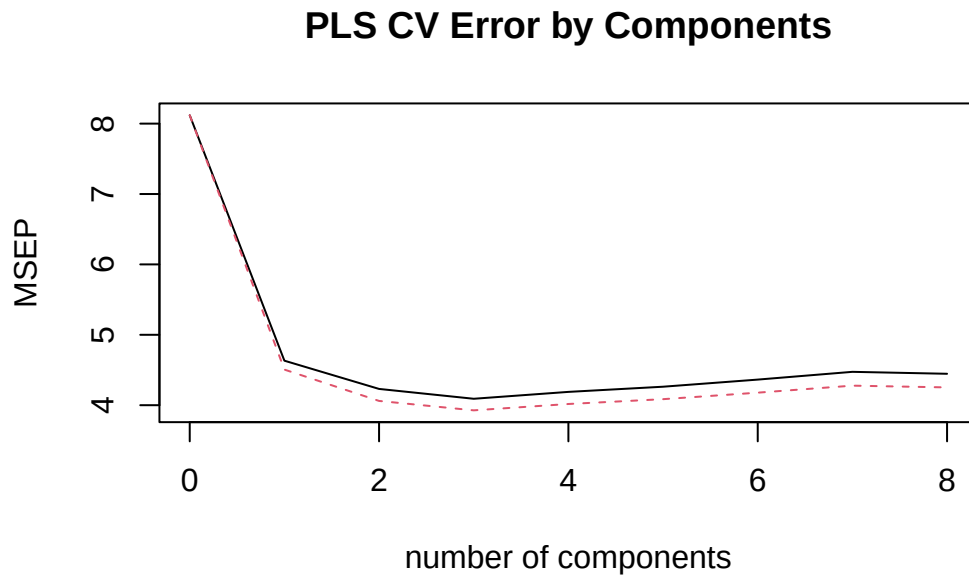
The PCR regression model achieved an R^2 of 0.703.

PLS

```
mali_df = as.data.frame(mali)

pls_fit = plsr(cotton ~ .,
               data = mali_df,
               validation = "CV",
               scale = TRUE)

validationplot(pls_fit, val.type = "MSEP",
               main = "PLS CV Error by Components",
               legend = FALSE)
```



The validation plot shows the CV and adjusted CV MSEP for each number of PLS components. We are not too concerned with the adjust CV MSEP, which adjusts for bias from CV and is very close to the black line. The CV error drops sharply from 0 to 1 component and levels off after 2 components, suggesting that 2 components is a reasonable choice.

```
best_ncomp = 2

pls_coef = coef(pls_fit, ncomp = best_ncomp, intercept = TRUE)

yhat_pls = predict(pls_fit, ncomp = best_ncomp, newdata = mali_df)
SSE_pls = sum((y_raw - yhat_pls)^2)
r2_pls = 1 - SSE_pls / SST
```

The PLS model achieved an R^2 of 0.745.

Summary

We now compare all the models R^2 to assess which model has the best fit.

```
#summary metrics
table = data.frame(
  Method = c("Multiple Regression", "PCR (m = 3)", paste0("PLS (m = 2)")),
  R_Squared = c(r2_mr, r2_pcr, r2_pls)
)
kable(table, digits = 4,
  caption = "Comparison of Model Fit Quality for Multiple Regression, PCR and PLS.")
```

Method	R_Squared
Multiple Regression	0.7739
PCR (m = 3)	0.7034
PLS (m = 2)	0.7450

Table 9: Comparison of Model Fit Quality for Multiple Regression, PCR and PLS.

MLR achieved the highest R^2 of 0.7739, meaning that 77.39% of the variance in cotton planted is explained by the model. However, MLR uses all predictors directly, which can cause problems when predictors are correlated with each other. PCR achieved an R^2 of 0.7034 using 3 components. This is slightly lower than MLR, but PCR handles correlated predictors better by combining them into a smaller set of uncorrelated components. The small drop in R^2 is the price paid for a simpler and more stable model. PLS achieved an R^2 of 0.7450 using only 2 components, which is better than PCR despite using fewer components. This is because PLS builds its components with the response variable in mind, making it better at predicting cotton yield than PCR. Overall, MLR explains the most variance but struggles when predictors are correlated. PLS gives a good balance between predictive performance and simplicity, and performs better than PCR here because its components are built to be useful for prediction rather than just summarising the predictors.